

```

# Program to generate and plot look-up tables
# Made by: Neo Vorsatz

import numpy as np
from matplotlib import pyplot as plt
from scipy.io import wavfile

F_SIGNAL = 0.5
PLOT = "drum"
NS = 1024
MAX = 4095

#generate sine LUT
lut = []
file = open(file="sine_lut.txt", mode="w")
file.write("{")
for t in range(NS):
    val = round((MAX/2)*(1+np.sin(2*np.pi*(t/NS))))
    lut.append(val)
    file.write(str(val))
    if t==NS-1:
        file.write("}")
    else:
        file.write(",")
file.close()

if PLOT=="sine":
    plt.plot(range(NS), lut, color="black")
    plt.xlabel("Time")
    plt.ylabel("Amplitude")
    plt.title("Sine Wave LUT")
    plt.show()

#generate sawtooth LUT
lut = []
file = open(file="sawtooth_lut.txt", mode="w")
file.write("{")
for t in range(NS):
    val = round(t*(MAX/NS))
    lut.append(val)
    file.write(str(val))
    if t==NS-1:
        file.write("}")
    else:
        file.write(",")
file.close()

if PLOT=="sawtooth":
    plt.plot(range(NS), lut, color="black")
    plt.xlabel("Time")
    plt.ylabel("Amplitude")
    plt.title("Sawtooth Wave LUT")
    plt.show()

```

```

#generate triangular LUT
lut = []
file = open(file="triangular_lut.txt", mode="w")
file.write("{")
for t in range(NS):
    if t<NS/2:
        val = round(2*t*(MAX/NS))
    else:
        val = round(MAX-2*(t-NS/2)*(MAX/NS))
    lut.append(val)
    file.write(str(val))
    if t==NS-1:
        file.write("}")
    else:
        file.write(",")
file.close()

if PLOT=="triangular":
    plt.plot(range(NS), lut, color="black")
    plt.xlabel("Time")
    plt.ylabel("Amplitude")
    plt.title("Triangular Wave LUT")
    plt.show()

#generate LUT from WAV
def wav_to_lut(path:str) -> list:
    """
    Takes the file path to a .wav file, and returns the LUT
    """
    #calculate the number of samples
    frequency = 44100 #Hertz
    desired_period = 2 #seconds
    cropped_samples = frequency*desired_period #number of samples
    #extract the desired number of data points
    sample_rate, data = wavfile.read(path)
    if data.ndim > 1:
        data = data[:, 0] #get only the first channel
    data = data[:cropped_samples] #extracting the first few samples
    data = data.astype(np.float32) #convert type
    indices = np.linspace(0, len(data)-1, NS).astype(int)
    lut = data[indices] #extract 128 values

    #normalise and round
    lut -= np.min(lut) #shift lowest value to 0
    lut *= MAX/np.max(lut) #normalise to the maximum
    lut = lut.astype(int)

    return lut

#generate piano LUT
lut = wav_to_lut("WAV_FILES/piano.wav")
file = open(file="piano_lut.txt", mode="w")
file.write("{")

```

```

for t in range(NS):
    file.write(str(lut[t]))
    if t==NS-1:
        file.write("}")
    else:
        file.write(",")
file.close()

if PLOT=="piano":
    plt.plot(range(NS), lut, color="black")
    plt.xlabel("Time")
    plt.ylabel("Amplitude")
    plt.title("Piano LUT")
    plt.show()

#generate guitar LUT
lut = wav_to_lut("WAV_FILES/guitar.wav")
file = open(file="guitar_lut.txt", mode="w")
file.write("{")
for t in range(NS):
    file.write(str(lut[t]))
    if t==NS-1:
        file.write("}")
    else:
        file.write(",")
file.close()

if PLOT=="guitar":
    plt.plot(range(NS), lut, color="black")
    plt.xlabel("Time")
    plt.ylabel("Amplitude")
    plt.title("Guitar LUT")
    plt.show()

#generate drum LUT
lut = wav_to_lut("WAV_FILES/drum.wav")
file = open(file="drum_lut.txt", mode="w")
file.write("{")
for t in range(NS):
    file.write(str(lut[t]))
    if t==NS-1:
        file.write("}")
    else:
        file.write(",")
file.close()

if PLOT=="drum":
    plt.plot(range(NS), lut, color="black")
    plt.xlabel("Time")
    plt.ylabel("Amplitude")
    plt.title("Drum LUT")
    plt.show()

```